

RESTFUL API trong Flutter với HTTP và Dio

Sinh viên: Trần Viết Thắng

Sinh viên: Trần Duy Nguyên

Khoa điện tử viễn thông
Đại học bách khoa
Đại học Đà Nẵng

Ngày 27 tháng 10 năm 2025



Outline

- ① Tổng quan về API
- ② RESTFUL API trong Flutter
 - Thư viện HTTP
 - Thư viện Dio
- ③ Dio vs HTTP
- ④ Dio Features
 - Authentication with interceptor
 - Xử lý lỗi và Cơ chế retry



Table of Contents

- ① Tổng quan về API
- ② RESTFUL API trong Flutter
 - Thư viện HTTP
 - Thư viện Dio
- ③ Dio vs HTTP
- ④ Dio Features
 - Authentication with interceptor
 - Xử lý lỗi và Cơ chế retry



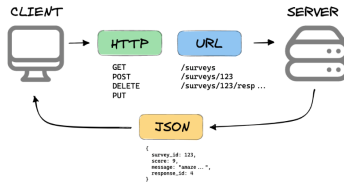
Gọi API là gì ?

- Các dữ liệu lớn thường được lưu trữ ở các server trên Internet
- Các chương trình ở local host không thể trực tiếp truy cập mà phải thông qua các **API** từ phía server.
- API được viết tắt tiếng Anh : Application Programming Interface. Cho phép người dùng gửi các yêu cầu để đọc hoặc ghi dữ liệu
- Các dữ liệu trả về từ API thường ở dạng JSON.



Giới thiệu về RESTful API

- RESTful API (Representational State Transfer API) là một phong cách kiến trúc và phương pháp tiếp cận được sử dụng rộng rãi trong việc xây dựng các dịch vụ web.
- RESTful API dựa trên giao thức HTTP để thực hiện các hoạt động CRUD (Create, Read, Update, Delete) trên các tài nguyên.
- RESTful API theo tổ chức mô hình Client-Server



Các phương thức hỗ trợ phổ biến

Phương thức	Công dụng	Chú ý
GET	lấy dữ liệu	yêu cầu dữ liệu từ server
DELETE	xóa dữ liệu	xóa dữ liệu dựa trên yêu cầu
POST	gửi dữ liệu mới	Tạo một bản ghi mới cho phép trùng lập
PUT	Cập nhập dữ liệu	Xóa dữ liệu cũ Thay thế bằng dữ liệu mới
PATCH	Cập nhật một phần	Chỉ cập nhật một phần không xóa hoàn toàn



Table of Contents

- ① Tổng quan về API
- ② RESTFUL API trong Flutter
 - Thư viện HTTP
 - Thư viện Dio
- ③ Dio vs HTTP
- ④ Dio Features
 - Authentication with interceptor
 - Xử lý lỗi và Cơ chế retry



Sơ lược

- Để có thể thực hiện các yêu cầu về HTTP, Flutter hỗ trợ 2 thư viện :
 - Truyền thống : *http* → khá phổ biến, dễ xài.
 - Nâng cao : *dio* → hỗ trợ nhiều tính năng hơn.
- Lập trình bất đồng bộ với **Future** và **async/await** (Các lời gọi API cần có thời gian truyền và nhận).
- Kỹ thuật để chuyển đổi dữ liệu dạng JSON về các đối tượng trong dart.
- Vì dữ liệu được trả về từ HTTP không có cấu trúc rõ ràng, chúng ta nên chuyển đổi về các đối tượng rõ ràng để dễ dàng xử lí.



Cài đặt gói

- Cài đặt gói trong tệp pubspec.yaml

```
1 dependencies:  
2   flutter:  
3     sdk: flutter  
4   dio: ^5.0.0  
5   http: ^1.5.0  
6
```

- Thêm các gói vào chương trình

```
1 import 'package:dio/dio.dart';  
2 import 'package:http/http.dart' as http;  
3
```

- Thêm gói để chuyển đổi từ định dạng JSON:

```
1 import 'dart:convert';
```



HTTP GET request

```
1 class RemoteServices {  
2   final String _Url = 'https://jsonplaceholder.typicode.com/  
   posts';  
3   Future<List<Post>> getPost() async {  
4     try {  
5       final uri = Uri.parse(_Url);  
6       final response = await http.get(uri).timeout(const Duration  
         (seconds: 10));  
7       if (response.statusCode == 200) {  
8         final List<dynamic> jsonData = jsonDecode(response.body);  
9         return jsonData.map((e) => Post.fromJson(e)).toList  
10        ();  
11      } else {  
12        throw Exception('Server error: ${response.statusCode}')  
13      };  
14    }  
15  }
```



Ứng dụng lấy bài viết từ JSONplaceholder

- Trường dữ liệu trong jsonplaceholder/post:

```
1 {  
2   "userId": 1,  
3   "id": 1,  
4   "title": "Title 1",  
5   "body": "Hello World"  
6 },
```

- Tạo lớp **Post** giúp chúng ta thao tác dễ dàng hơn:

```
1 class Post {  
2   int userId;  
3   int id;  
4   String title;  
5   String ?body;  
6   Post({  
7     required this.userId,  
8     required this.id,  
9     required this.title,  
10    required this.body,  
11    });  
12 }  
13
```



Chạy ứng dụng

Posts

1

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam

2

qui est esse

est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis

3

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
voluptatem doloribus vel accusantium quis pariatur

4

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
quis hic commodi nesciunt rem tenetur doloremque ipsam iure



HTTP POST

- Để có thể sử dụng phương thức post, ta cần phải có phần "payload" (dữ liệu cần tải lên) và phần headers.
- Phần headers thường là một cặp khóa giá trị.

```
1 Future<bool> addBook(Book book) async {  
2   try {  
3     final response = await http.post(  
4       Uri.parse(url),  
5       headers: {'Content-Type': 'application/json'},  
6       body: json.encode(book.toJson()),  
7     );  
8     if (response.statusCode == 200 || response.statusCode == 201) {  
9       debugPrint("Book added successfully");  
10      return true;  
11    } else { throw Exception('Failed to add book'); }  
12    } catch (e) {  
13      debugPrint("Error adding book: $e");  
14      return false;  
15    }  
16  }
```



HTTP PUT

- Giống với phương thức post, ta cũng cần có phần payload và phần header.
- Ta cần xác định cần phần tử muốn thay đổi, thông thường qua các địa chỉ id được máy chủ cấp, nó là trường *id* ở trong định dạng JASON.

```
1 Future<bool> updateBook(Book book) async {  
2   try {  
3     final response = await http.put(  
4       Uri.parse('$url/${book.id}'),  
5       headers: {'Content-Type': 'application/json'},  
6       body: json.encode(book.toJson()),  
7     );  
8     if (response.statusCode == 200) {  
9       debugPrint("Book updated successfully"); return true;  
10    } else {  
11      throw Exception('Failed to update book');  
12    }  
13  } catch (e) {  
14    debugPrint("Error updating book: $e"); return false;  
15  }  
16 }
```



HTTP DELETE

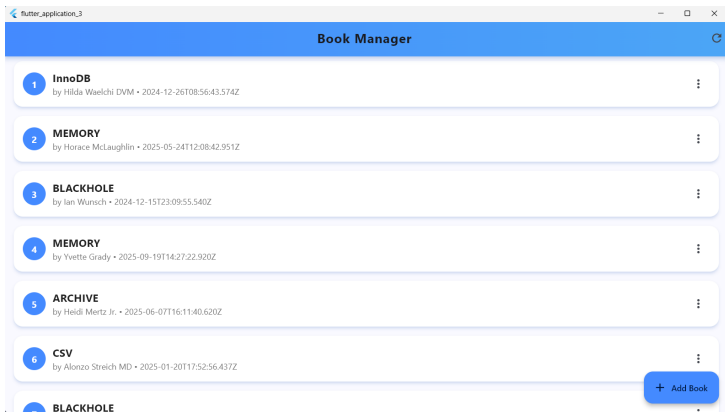
- Để xóa 1 phần tử, ta chỉ cần xác định địa chỉ *id* của phần tử cần xóa.

```
1 Future<bool> deleteBook(String id) async {  
2   try {  
3     final response = await http.delete(Uri.parse('$url/$id'))  
4     );  
5     if (response.statusCode == 200 || response.statusCode ==  
6       204) {  
7       debugPrint("Book deleted successfully");  
8       return true;  
9     } else {  
10      throw Exception('Failed to delete book');  
11    }  
12    } catch (e) {  
13      debugPrint("Error deleting book: $e");  
14      return false;  
15    }  
16  }
```



Thư viện HTTP

Áp dụng: Ứng dụng quản lí sách



Giới thiệu về Thư Viện Dio

- Dio là một HTTP Client mạnh mẽ cho Dart, hỗ trợ Interceptors (trình chặn), cấu hình toàn cục, FormData,.. và dễ sử dụng.
 - Dio giúp việc xử lý mạng trở nên đơn giản và quản lý tốt nhiều tình huống đặc biệt.
 - Nó giúp bạn dễ dàng xử lý nhiều yêu cầu mạng chạy đồng thời, đồng thời đảm bảo tính an toàn nhờ cơ chế xử lý lỗi tiên tiến.



DIO GET request

```
1 List<Post> _posts = [];  
2 bool _isLoading = false;  
3 Future<void> fetchData() async {  
4   setState(() {  
5     _isLoading = true;  
6   });  
7   try {  
8     final dio = Dio();  
9     final response = await dio.get('https://jsonplaceholder.typicode.com/posts  
10   ');  
11     if (response.statusCode == 200) {  
12       final List data = response.data;  
13       _posts = data.map((json) => Post.fromJson(json)).toList();  
14     } else {  
15       throw Exception('Failed_to_load_posts');  
16     }  
17     catch (e) {  
18       debugPrint('Error:$e');  
19     }  
20     setState(() {  
21       _isLoading = false;  
22     });  
23   }  
}
```



Chạy ứng dụng

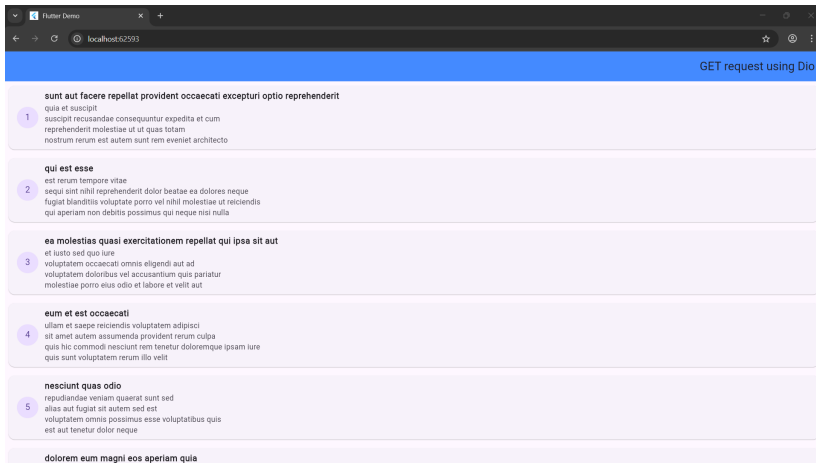


Table of Contents

- ① Tổng quan về API
- ② RESTFUL API trong Flutter
 - Thư viện HTTP
 - Thư viện Dio
- ③ Dio vs HTTP
- ④ Dio Features
 - Authentication with interceptor
 - Xử lý lỗi và Cơ chế retry



Tiêu chí	HTTP	Dio
Mức độ phức tạp / Hiệu năng	Nhẹ, đơn giản, phù hợp dự án nhỏ; ít overhead, hiệu năng ổn định cho tác vụ đơn.	Mạnh mẽ, có nhiều tính năng, tối ưu cho dự án lớn; xử lý nhiều request song song hiệu quả.
Xử lý JSON	Phải tự decode thủ công bằng <code>jsonDecode</code> .	Tự động decode JSON nhanh và tiện lợi.
Cấu hình toàn cục	Không có — phải cấu hình trong từng request riêng lẻ.	Hỗ trợ qua <code>BaseOptions</code> , giúp giảm trể và đồng bộ tốt hơn.
Interceptors	Không hỗ trợ, phải viết thủ công nếu cần can thiệp.	Có sẵn, giúp logging và xử lý pipeline hiệu quả hơn.
Upload / Download hiệu năng cao	Upload dùng <code>MultipartRequest</code> , khó quản lý và không hỗ trợ theo dõi tiến trình.	Có <code>FormData</code> và <code>onReceiveProgress</code> , hỗ trợ theo dõi tiến trình và tối ưu tốc độ I/O.



Table of Contents

- 1 Tổng quan về API
- 2 RESTFUL API trong Flutter
 - Thư viện HTTP
 - Thư viện Dio
- 3 Dio vs HTTP
- 4 Dio Features
 - Authentication with interceptor
 - Xử lý lỗi và Cơ chế retry



Interceptors

- Interceptors là một cơ chế mạnh mẽ cho phép giám sát, chỉnh sửa và thực hiện lại (retry) các lời gọi API.
- Interceptors trong Dio cho phép bạn chặn và chỉnh sửa các request hoặc response ở phạm vi toàn cục trước khi chúng được gửi đi hoặc nhận về.



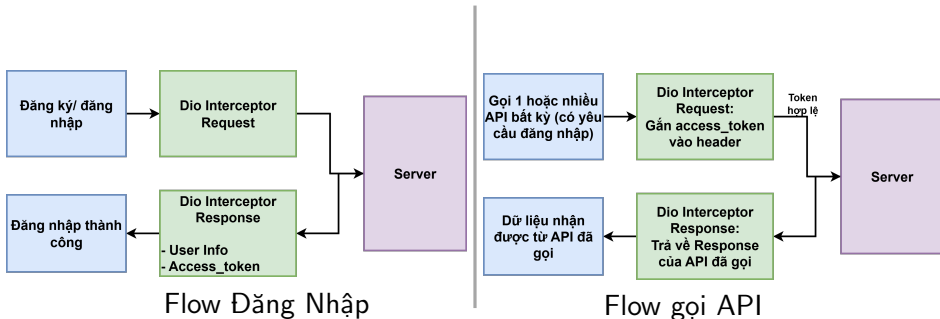
Authentication

- Để một ứng dụng có thể hoạt động với cơ chế đăng ký và đăng nhập, điều quan trọng cần quan tâm là cơ chế Authentication (xác thực người dùng).
- Khi người dùng đăng ký hoặc đăng nhập thành công, server sẽ trả về một token. Token này giúp người dùng được xác thực và cấp quyền truy cập vào các tài nguyên hoặc thao tác yêu cầu cao hơn.



Authentication with interceptor

Login Authentication và API Request



Các thành phần chính của Dio Interceptor

- **onRequest(RequestOptions options):** được gọi trước khi gửi yêu cầu đến server, dùng để chỉnh sửa hoặc bổ sung thông tin cho request (ví dụ: chèn token xác thực).
 - Ví dụ chèn access_token vào header trước mỗi request:

```
1 dio.interceptors.add(InterceptorsWrapper(  
2   onRequest: (options, handler) {  
3     options.headers["Authorization"] = "Bearer $access_token  
4     ";  
5     return handler.next(options);  
6   },  
7 ));
```



Các thành phần chính của Dio Interceptor (Con't)

- **onResponse(Response response):** được gọi khi server trả về phản hồi thành công.
- Dùng để xử lý dữ liệu phản hồi, ví dụ: lưu lại access_token sau khi đăng nhập.

```
1 dio.interceptors.add(InterceptorsWrapper(  
2   onResponse: (response, handler) {  
3     if (response.requestOptions.path.contains("/login")) {  
4       final token = response.data["access_token"];  
5       if (token != null) {  
6         saveToken(access_token);  
7         print("Token saved: $access_token");  
8       }  
9     }  
10    return handler.next(response);  
11  },  
12  ));  
13
```



Các vấn đề thực tiễn:

- Thất bại trong việc yêu cầu API từ server.
- Các vấn đề về kết nối và mạng.
- Thời gian gửi và nhận một yêu cầu quá lâu, hiện tượng timeout ở server.
- Lỗi liên quan đến HTTP ở 2 phía server và client.
- Gói **dio** cung cấp cho ta những tính năng mạnh mẽ để có thể xử lý được những lỗi trên dễ dàng :
 - Interceptors (chặn và sửa đổi request/response)
 - Error handling có cấu trúc
 - Timeout, cancel request
 - Retry mechanism (thử lại request khi lỗi mạng)



Error Handling

- Có lỗi xảy ra trong quá trình xử lý, gói *dio* sẽ tạo ra đối tượng kiểu **DioException**. Thuộc tính quan trọng **type**

DioExceptionType	Mô tả
connectionTimeout	quá thời gian kết nối.
sendTimeout	Gửi request quá lâu (timeout).
receiveTimeout	Nhận phản hồi quá lâu (timeout).
badResponse	Server trả về mã lỗi (4xx, 5xx).
cancel	Request bị hủy
connectionError	Không thể kết nối .
unknown	Lỗi không xác định.



Xử lý lỗi

- Ta nên nhóm các Exception lại để dễ quản lý:

```
1 String _getErrorMessage(DioException error) {  
2   switch (error.type) {  
3     case DioExceptionType.connectionTimeout:  
4       return 'Connection timeout - please check your internet';  
5     case DioExceptionType.sendTimeout:  
6       return 'Send timeout - request took too long';  
7     case DioExceptionType.receiveTimeout:  
8       return 'Receive timeout - server is not responding';  
9     case DioExceptionType.badResponse:  
10      return 'Server error: ${error.response?.statusCode} - ${error.response?.  
statusMessage}';  
11     case DioExceptionType.cancel:  
12       return 'Request was cancelled';  
13     case DioExceptionType.connectionError:  
14       return 'No internet connection. Please check your network';  
15     case DioExceptionType.badCertificate:  
16       return 'Certificate verification failed';  
17     case DioExceptionType.unknown:  
18     default:  
19       return 'Unexpected error occurred. Please try again';  
20   }
```



Xử lý lỗi

- Sử dụng *try-catch* để xử lý các lỗi:

```
1  try {  
2    final response = await _dio.get('');  
3  } on DioException catch (e) {  
4    String errorMsg = _getErrorMessage(e);  
5    if (e.type == DioExceptionType.connectionError) {  
6      errorMsg = "No internet connection available";  
7    } else if (e.response?.statusCode == 404) {  
8      errorMsg = "The server not found";  
9    }  
10   debugPrint("Error getting books: $errorMsg");  
11   return ServiceResult(success: false, errorMessage: errorMsg);  
12 } catch (e) {  
13   debugPrint("Unexpected error getting books: $e");  
14   return ServiceResult(  
15     success: false,  
16     errorMessage: 'An unexpected error occurred',  
17   );  
18 }
```



Cơ chế Retry

- Cơ chế retry của gói dio cho phép ta yêu cầu lại các request từ server một cách tự động.
- Hiệu quả khi:
 - Lỗi mạng tạm thời.
 - Lỗi có thể khắc phục nhanh.
- Tránh ứng dụng báo lỗi quá nhanh tới người dùng → giảm trải nghiệm.
- Để sử dụng cơ chế này ta có 2 cách:
 - Sử dụng gói **Dio Smart Retry** được xây dựng sẵn (dễ dàng).
 - Tự xây dựng cơ chế retry từ lớp **Interceptor**.



Cơ chế Retry

1 Ta thêm gói vào pubspec.yaml :

```
1 dependencies:  
2   dio_smart_retry: ^7.0.0  
3
```

2 Thêm gói vào tệp chương trình :

```
1 import 'package:dio_smart_retry/dio_smart_retry.dart';  
2
```

```
1 final dio = Dio();  
2 dio.interceptors.add(RetryInterceptor(  
3   dio: dio,  
4   logPrint: print, // (optional)  
5   retries: 1, // (optional)  
6   retryDelays: const [  
7     Duration(seconds: 1),  
8   ], ));
```



THANK YOU FOR LISTENING

